

Visão Geral

Abaixo, temos informações sobre a visão geral do sistema descrito:

- **Objetivo do Sistema:** Facilitar a organização e atribuição de atividades em equipes, permitindo que múltiplas tarefas sejam gerenciadas e vinculadas a diversos usuários simultaneamente (relacionamento muitos-para-muitos)
- **Contexto de Uso:** O sistema funciona como um backend de gerenciamento onde administradores e usuários podem criar, atualizar e monitorar o progresso de tarefas, utilizando autenticação segura para controle de acesso

Decisões Arquiteturais

Abaixo, podemos revisar as decisões arquiteturais decididas pelo grupo.

- **Escolha da Arquitetura Hexagonal (Ports and Adapters):** A principal justificativa é o **desacoplamento total** entre a lógica de negócio (Core) e as tecnologias externas (Banco de Dados, API). Isso permite que o sistema troque, por exemplo, o banco de dados MySQL por um MongoDB sem alterar a regra de negócio central
- **Padrões Aplicados:**
 - **Injeção de Dependência:** Utilizada no arquivo main.py para instanciar repositórios e serviços, garantindo que o núcleo da aplicação receba suas dependências já configuradas.
 - **Repository Pattern:** Define portas (Protocols) que estabelecem o contrato de comunicação com o banco de dados, implementados por adaptadores específicos (MySQL).
 - **DTO (Data Transfer Objects):** Utilização de classes Pydantic para validar e tipar os dados que entram e saem da API

Justificativas Técnicas

A seguir, podemos obter informações sobre as decisões relacionadas a linguagem Python e MySQL para realizar o trabalho:

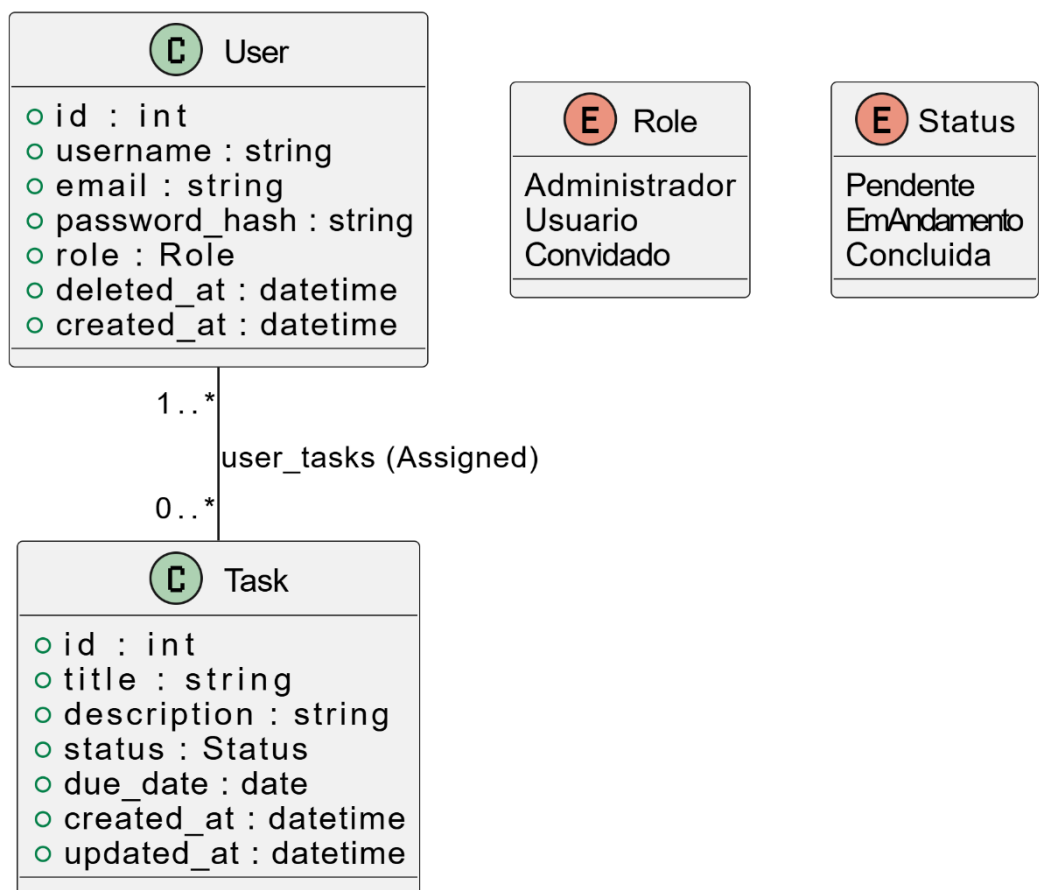
- **Python:** É a linguagem central utilizada para implementar uma **Arquitetura Hexagonal** robusta através do framework **FastAPI** e servidor Uvicorn, estruturando a lógica de negócio em serviços de domínio desacoplados da infraestrutura por meio de

Ports (Protocols) e garantindo segurança com autenticação JWT e validação de dados via Pydantic

- **SQL (MySQL):** Define o banco de dados `task_manager` com um esquema que suporta relações muitos-para-muitos entre usuários e tarefas, assegurando a integridade dos dados com chaves estrangeiras e tipos ENUM, além de otimizar a performance por meio de índices e suportar exclusão lógica (*soft delete*)

Modelagem de Dados

Na modelagem de dados, foi utilizado o padrão UML para a representação da Modelagem do nosso sistema:



Fluxo de Requisições

API de utilização é JWT para autenticação e organiza os endpoints em categorias de Autenticação, Usuários e Tarefas. Abaixo, temos mais detalhadamente:

- **Autenticação:**

- POST /auth/login: Recebe e-mail e senha, retornando um token de acesso.
- POST /auth/logout: Invalida o token atual através de uma *blacklist*.
- **Gestão de Usuários:**
 - POST /users: Criação de novo usuário com definição de cargo (Administrador, Usuário, Convidado).
 - GET /users/{user_id}: Recuperação de dados de perfil (requer autenticação).
- **Gestão de Tarefas:**
 - POST /tasks: Criação de tarefa permitindo a atribuição imediata de uma lista de user_ids.
 - GET /tasks?assignedTo={id}: Filtra todas as tarefas vinculadas a um colaborador específico.
 - PUT /tasks/{task_id}: Atualiza status (pendente, em andamento, concluída) ou detalhes da tarefa.

Configuração e Deploy

Abaixo temos informações sobre a configuração e o Deploy:

- **Guia de Execução:**
 1. Prepare o ambiente MySQL e execute o comando de criação do banco de dados.
 2. No arquivo main.py, o sistema orquestra a inicialização: instancia os adaptadores de infraestrutura, injeta-os nos serviços de domínio e inicia o servidor.
 3. Execute o projeto via terminal:
 4. O servidor será hospedado em `http://127.0.0.1:8000`, com a interface interativa disponível em `/docs`.
- **Configuração do Ambiente:** A conexão é gerenciada pelo SQLAlchemy através do engine, utilizando o driver pymysql para comunicação com o MySQL

Instruções Importantes

- Antes de executar qualquer coisa, certifique-se de possuir o python 3 instalado no seu computador:
 - No Windows, abra o CMD usando as teclas Windows + R e digitando 'cmd'.
Digite o seguinte comando: `python --version`
- Na root do projeto, abra o terminal e digite o seguinte comando python:
 - `py -m venv .venv`
- Isso irá criar seu ambiente virtual.
 - Ative seu ambiente virtual usando `.\venv\Scripts\activate`
- Finalmente, com o venv ativado no terminal, digite:
 - `'pip install -r requirements.txt'`

Como Realizar os Testes

- Após instalar todas as dependências, basta abrir o terminal e rodar o(s) seguinte(s) comando(s):
 - Para testes globais, rode o seguinte comando:
 - `py -m pytest`
 - Para testes unitários, rode o seguinte comando:
 - `pytest .\tests\unit\[nome_do_arquivo.py]`